Name:- Sanvi Wadhankar
SEM:- 4TH
SEC:- C
PRN:- 24070521191
PRACTICAL 12

onecompiler.com/plsql/44jqej8p4

queries.sql 44jqej8p4

```oracle
CREATE table Employees(
  employee_id NUMBER,
  name varchar2(20)
);

insert INTO Employees values(101,'Manjiri');
insert into Employees values(101,'Masti');
commit;

CREATE OR REPLACE PROCEDURE get_employee_name(p_emp_id
NUMBER)
IS
v_name varchar2(20);
BEGIN
SELECT name INTO v_name FROM Employees
WHERE employee_id = p_emp_id;
DBMS_OUTPUT.PUT_LINE('NAME: ' || v_name);
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('Error: Employee not found');
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE('Error: Multiple records found');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Unexpected error occurred');
END;
/
SET SERVEROUTPUT ON;

BEGIN
    get_employee_name(101);
END;
/
```

STDIN
Input for the program (Optional)

Output: 122 ms

```
Error: Multiple records found
```

queries.sql 44jqej8p4

```oracle
DECLARE
num1 NUMBER := 10;
num2 NUMBER := 0;
result NUMBER;
BEGIN
result := num1 / num2;
DBMS_OUTPUT.PUT_LINE('Result: ' || result);
EXCEPTION
WHEN ZERO_DIVIDE THEN
DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed');
END;
```

STDIN
Input for the program (Optional)

Output: 114 ms

```
Error: Division by zero is not allowed
```

onecompiler.com/plsql/44jqej8p4

OneCompiler

queries.sql 44jqej8p4

```oracle-plsql
DECLARE
invalid_marks EXCEPTION;
marks NUMBER := 25;
BEGIN
IF marks < 40 THEN
RAISE invalid_marks;
END IF;
DBMS_OUTPUT.PUT_LINE('Pass');
EXCEPTION
WHEN invalid_marks THEN
DBMS_OUTPUT.PUT_LINE('Error: marks must be 40 or above');
END;
/
```

STDIN

Input for the program (Optional)

Output: 102 ms

```
Error: marks must be 40 or above
```

OneCompiler

queries.sql 44jqej8p4

```oracle-plsql
CREATE TABLE Bank_Account (
    acc_no NUMBER PRIMARY KEY,
    balance NUMBER
);
INSERT INTO Bank_Account VALUES (101, 5000);
COMMIT;
CREATE OR REPLACE PROCEDURE withdraw_money(
    p_acc_no IN NUMBER,
    p_amount IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    -- Get current balance
    SELECT balance INTO v_balance
    FROM Bank_Account
    WHERE acc_no = p_acc_no;

    -- Check condition
    IF p_amount > v_balance THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Balance!');
    ELSE
        UPDATE Bank_Account
        SET balance = balance - p_amount
        WHERE acc_no = p_acc_no;

        DBMS_OUTPUT.PUT_LINE('Withdrawal successful');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Account not found');
```

STDIN

Input for the program (Optional)

Output: 144 ms

```
ORA-20001: Insufficient Balance!
```

OneCompiler

queries.sql 44jqej8p4

```sql
IS
    v_balance NUMBER;
BEGIN
    -- Get current balance
    SELECT balance INTO v_balance
    FROM Bank_Account
    WHERE acc_no = p_acc_no;

    -- Check condition
    IF p_amount > v_balance THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Balance!');
    ELSE
        UPDATE Bank_Account
        SET balance = balance - p_amount
        WHERE acc_no = p_acc_no;

        DBMS_OUTPUT.PUT_LINE('Withdrawal successful');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Account not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
/
SET SERVEROUTPUT ON;

BEGIN
    withdraw_money(101, 6000);
END;
/
```